RA - Assignment 2: Balanced allocation strategies

Jairo El Yazidi Ríos
https://github.com/JairoRY/balanced-allocation

November 2025

1 Introduction

In this assignment, we perform a series of simulations to study different strategies to distribute balls in bins and observe how their behavior affects the load balance over time. Specifically, we implement and compare several allocation schemes under the same experimental conditions:

- The **d-choice scheme**, where each ball selects among d randomly chosen bins before being placed.
- The **$(1+\beta)$-choice scheme**, which combines random and multi-choice placements according to a tunable probability parameter β.
- The **partial information scheme**, which makes allocation decisions using only partial information based on load quantiles.

Our focus is on simulating these variants, tracking how the load distribution evolves, and comparing their performance in terms of load imbalance (gap) throughout the allocation process.

2 Methods

2.1 Experimental Setup

The experiments were implemented in C++ and for each configuration we simulate the allocation of up to $n = m^2$ balls into m bins. The default number of bins is $m = 500$, and each setup is repeated $T = 20$ times to estimate statistical variability.

We measure the *gap*, defined as:

$$G_n = \max_{1 \le i \le m} \{X_i(n) - \frac{n}{m}\},$$

where $X_i(n)$ is the load of bin i after n balls have been placed.

During simulation, the program records the state of the system at predefined *checkpoints* to capture the evolution of the load distribution under different workloads. Two scenarios are considered:

- **Light-loaded scenario:** the number of balls n increases gradually, one by one, from 1 to m.
- **Heavy-loaded scenario:** the number of balls n increases in larger steps, from m up to m^2, in increments of m.

At each checkpoint, relevant statistics such as the load imbalance (gap) and bin load distribution are recorded, allowing for detailed comparisons across allocation schemes and workload intensities.

2.2 Algorithms Tested

The program supports multiple allocation schemes through command-line options:

- `--d`: number of choices in the d-choice process.
- `--beta`: activates the $(1 + \beta)$-choice scheme and specifies the value of β.
- `--k`: activates the partial information scheme ($k = 1$ or 2).
- `--b`: activates the optional b-batched setting.

The parameters are not fully independent as some settings conceptually overlap. Specifically, if the desired allocation strategy involves partial information, the program uses the `--k` option, which internally compares between $d = 2$ bins but with limited information about their loads. Similarly, the $(1 + \beta)$-choice scheme (`--beta`) also relies on comparing between one or two bins, depending on the probability β.

To avoid ambiguity, we established a fixed priority order among these parameters: `--k` takes precedence over `--beta`, and both override `--d`. This means that if multiple options are specified, the program will execute only the highest-priority scheme according to this hierarchy.

2.3 Statistical Analysis

After all trials, the program computes the mean and standard deviation of the gap across repetitions for each checkpoint:

$$\overline{G_n} = \frac{1}{T}\sum_{t=1}^{T} G_n^{(t)}, \quad \sigma_{G_n} = \sqrt{\frac{1}{T-1}\sum_{t=1}^{T}(G_n^{(t)} - \overline{G_n})^2}$$

The aggregated results are written to a CSV file for further analysis and visualization.

3 Results

3.1 d-choice scheme

Figure 1 below presents the evolution of the average gap for the d-choice scheme in the one-choice and two-choice versions asked in the statement and an additional one where $d = 3$.

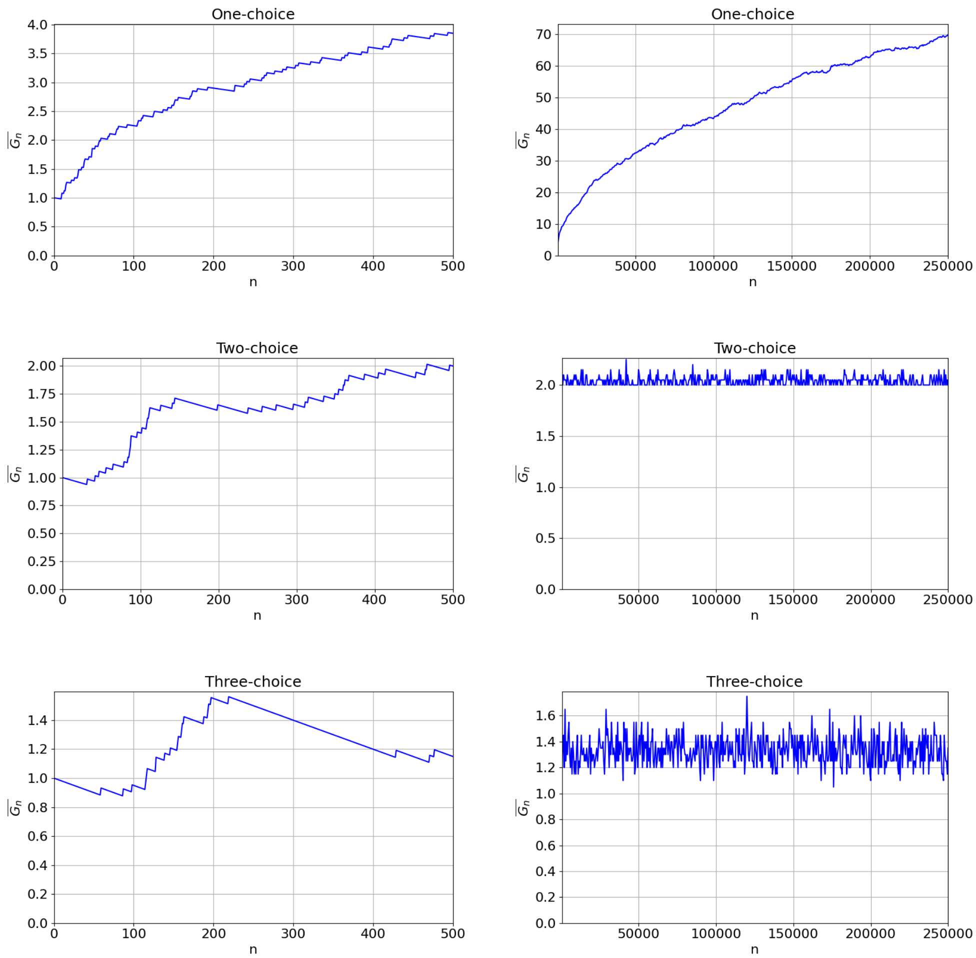

Figure 1: Evolution of the average gap for $d \in \{1, 2, 3\}$ in light-loaded (left) and heavy-loaded (right) scenarios.

In the light-loaded case, the curve for $d = 1$ shows a steady and almost linear growth of the average gap as more balls are inserted, indicating a continuously increasing imbalance. For $d = 2$, the curve also shows an apparently continuous growth, although a significant improvement is already seen, with the maximum gap reached being halved. Finally, for $d = 3$, we can see that in addition to reaching even lower maximum values, the gap even decreases. Thus, while the standard scheme exhibits unbounded growth within the observed range, the multi-choice ones approach a more stable load distribution.

In the heavy-loaded scenario, all curves start from higher initial values due to the larger number of balls added per step. The $d = 1$ configuration again displays continuous growth without a clear stabilization, while for $d = 2$ and $d = 3$ the average gap reaches a plateau, maintaining relatively constant values (around 2 in the two-choice and around 1.4 in the three-choice) as n grows. This stabilization reflects that the benefit of additional choices persists even at higher load intensities, although the difference between $d = 2$ and $d = 3$ is not extremely large, showing a saturation of improvement when increasing d further.

Standard deviation

Figure 2 illustrates the standard deviation of the gap for the same configurations.

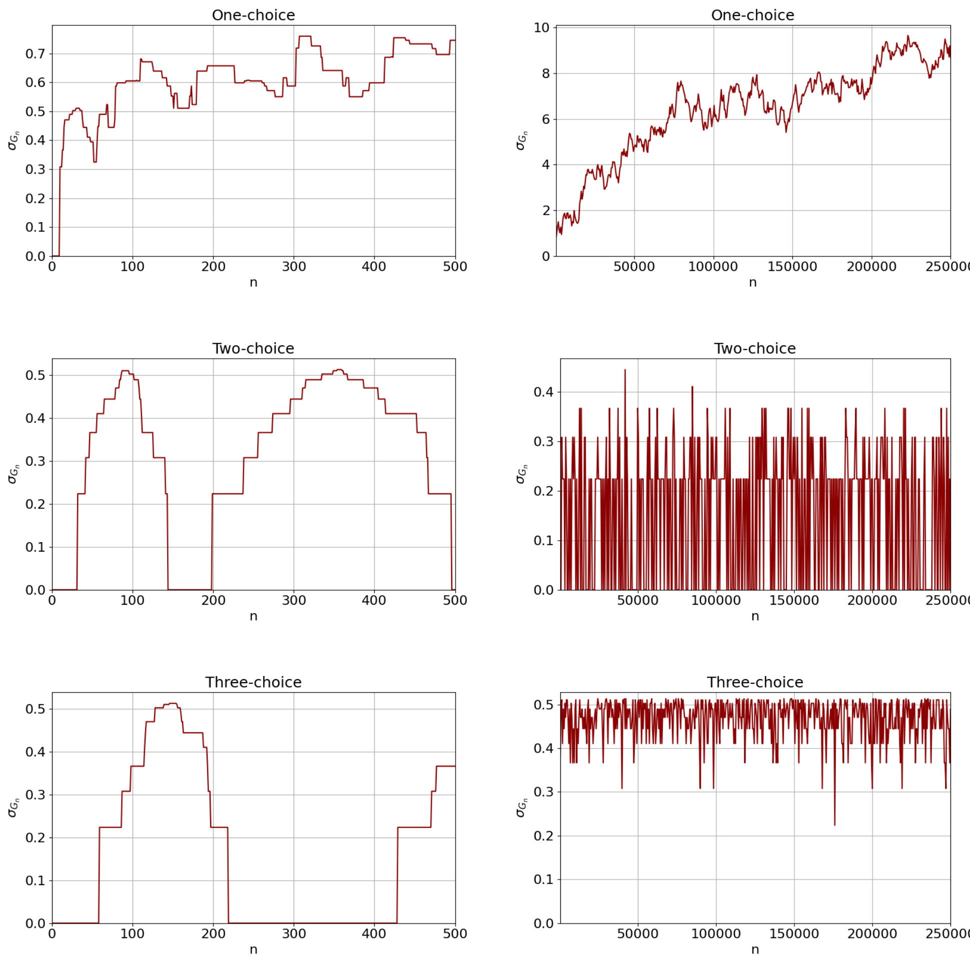

Figure 2: Evolution of the standard deviation of the gap for $d \in \{1, 2, 3\}$ in light-loaded (left) and heavy-loaded (right) scenarios.

For $d = 1$, the variability increases consistently with the number of balls, which means that the load imbalance fluctuates more strongly between repetitions. In contrast, for $d = 2$ and $d = 3$, the standard deviation constantly increases and falls without ever reaching too high values in any of the cases. The difference lies in the fact that in the two-choice scheme 0 is reached repeatedly, while for the case of $d = 3$ it does not happen, but the fluctuation is smaller. In any case, this behavior shows that the multi-choice schemes not only produce smaller average gaps but also more predictable results, as the system becomes less sensitive to random variation once it reaches a steady state.

3.2 $(1 + \beta)$-choice scheme

Figure 3 below shows the evolution of the average gap for the $(1 + \beta)$-choice scheme with values of β chosen to cover a balanced probability between the one-choice and two-choice schemes and an imbalance towards both options.

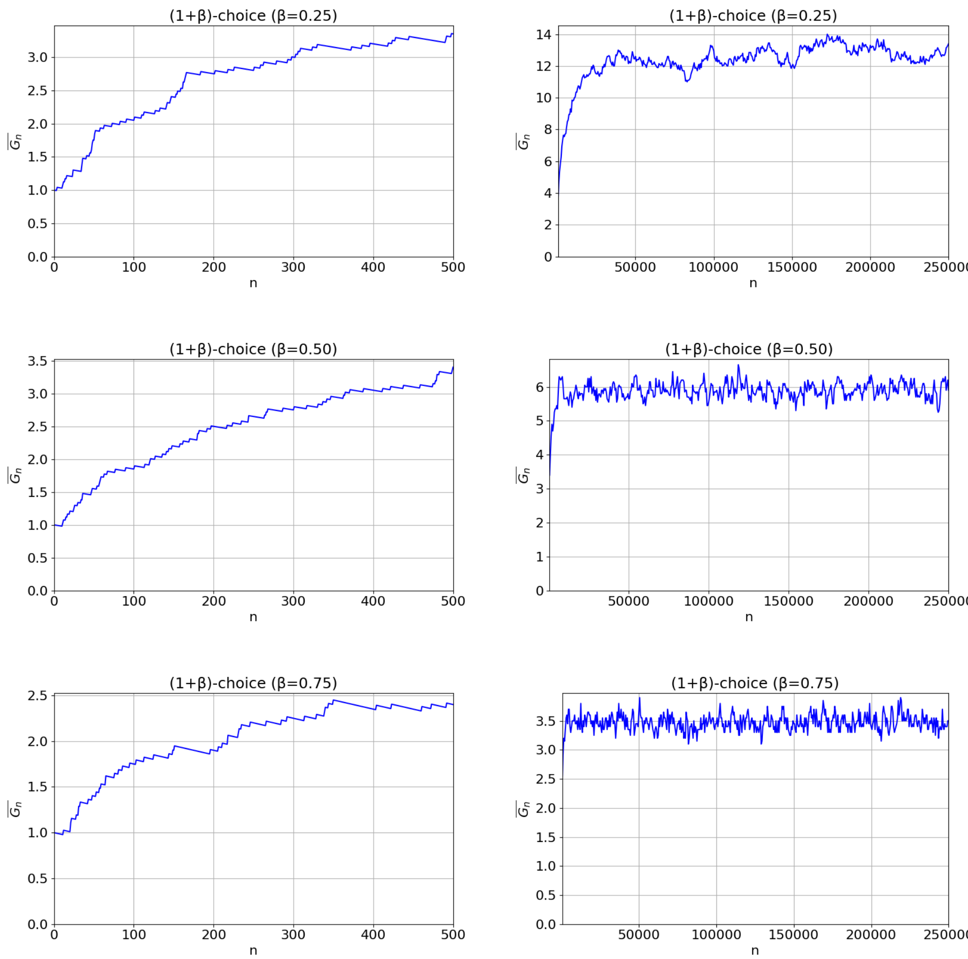

Figure 3: Evolution of the average gap for $\beta \in \{0.25, 0.5, 0.75\}$ in light-loaded (left) and heavy-loaded (right) scenarios.

In the light-loaded scenario, all curves begin with a constant rise, reflecting the early accumulation of load in the most occupied bins. However, as more balls are allocated, the curves tend to flatten and higher β values lead to lower steady-state gaps. For $\beta = 0.25$, the curve continues to increase slowly until the limit of $n = m$, whereas for $\beta = 0.5$ and very especially $\beta = 0.75$, the growth slows considerably and approaches a stable value, indicating that the system achieves nearly equilibrium when the two-choice process is applied more frequently.

In the heavy-loaded case, the same qualitative pattern is observed. All curves start at higher values due to the denser workload, then experience a rapid increase, and eventually tend toward stability as n increases. The higher-β configurations reach a plateau sooner and at a lower gap level (around 6 for $\beta = 0.5$ and around 3.5 for $\beta = 0.75$), while $\beta = 0.25$ continues to exhibit a moderate upward trend throughout. This shows that the influence of β on the balancing capacity of the system remains significant even in the heavy-loaded regime, although the differences between configurations narrow slightly as the system saturates.

Standard deviation

Figure 4 shows the standard deviation of the gap for the same values of β.

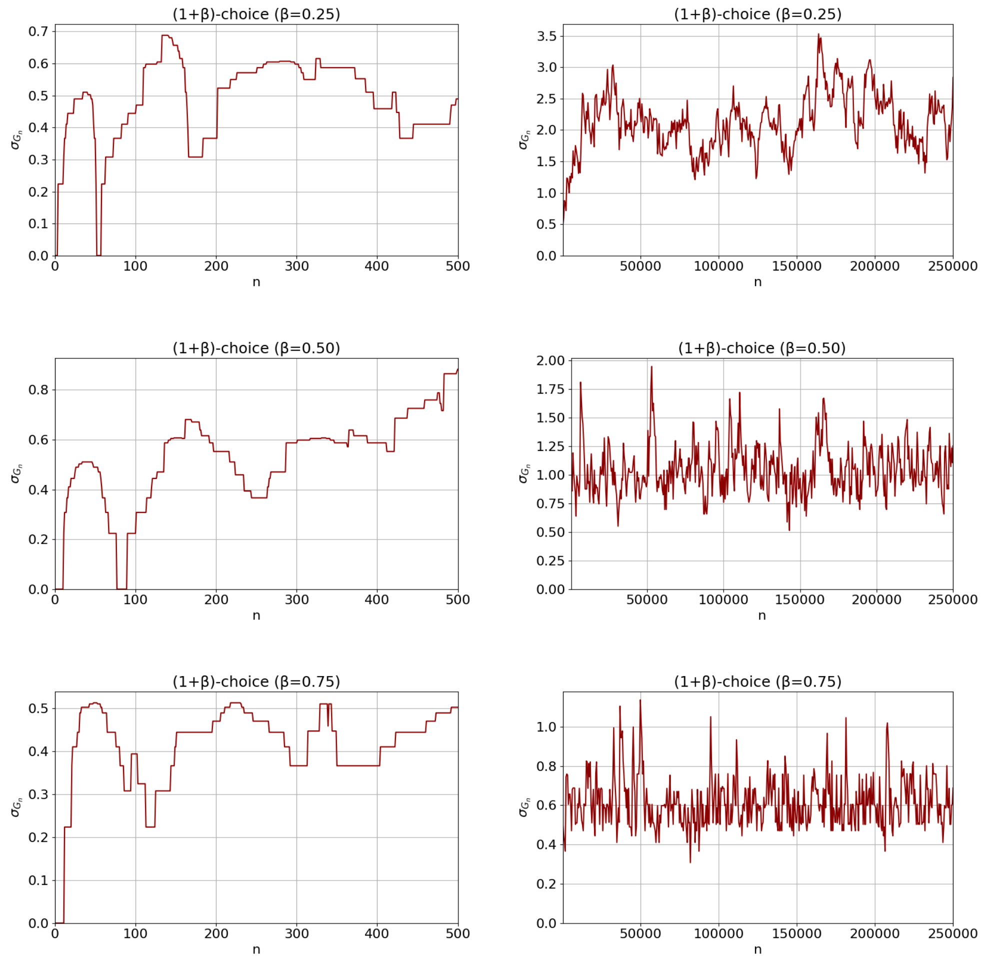

Figure 4: Evolution of the standard deviation of the gap for $\beta \in \{0.25, 0.5, 0.75\}$ in light-loaded (left) and heavy-loaded (right) scenarios.

Initially, the variability increases as the load increases, but then the curves stabilize, mirroring the behavior of the average gap. For $\beta = 0.25$, the standard deviation remains higher and continues to increase slightly, while for $\beta = 0.5$ and $\beta = 0.75$, it settles earlier at a nearly constant value. This indicates that higher β values not only improve balance, but also make the system's behavior more stable across repetitions, reducing fluctuations once the steady-state regime is reached.

3.3 b-batched setting

Figure 5 shows the evolution of the average gap for all configurations previously tested when the b-batched variant is applied. Since the interesting case is where b is "big", the analysis only takes place for the heavy-loaded scenario. The chosen value has been $b = m$ for all cases as a personal interpretation of the statement, because precisely when $b = m$ the values of n grow with increases of size m, so n takes the values $n = b$, $n = 2b$, $n = 3b, \ldots, n = m^2$.

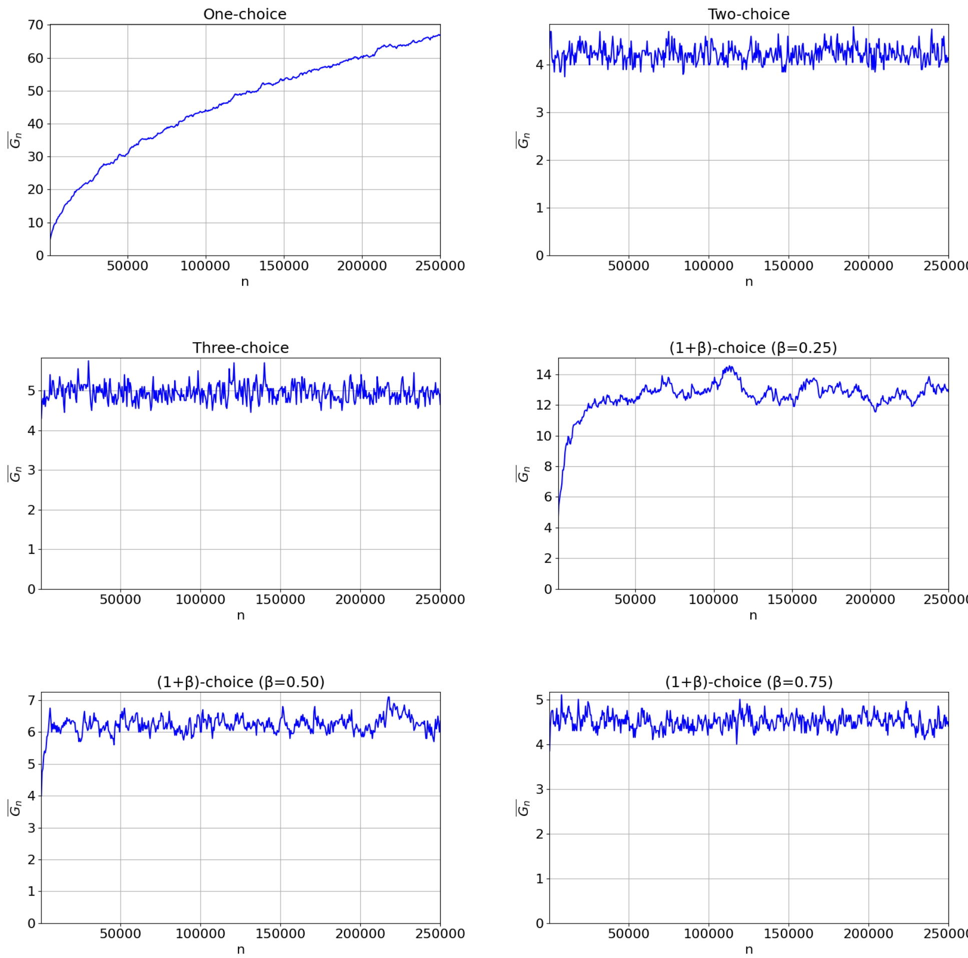

Figure 5: Evolution of the average gap for all previous configurations with b-batched setting.

Overall, the curves preserve the qualitative ordering observed in the non-batched heavy-loaded scenarios, although the magnitude of the gap is generally larger. This is expected because batching introduces a delay in how often the system reacts to changes in bin loads, causing the gap to increase more abruptly at each step. Nevertheless, the differences between schemes remain visible since the one-choice configuration exhibits the largest and most steadily increasing gap, while the two-choice and higher-β variants achieve significantly lower values and stabilize earlier.

A particularly notable phenomenon appears in the comparison between the two-choice and three-choice batched configurations. Contrary to the expected hierarchy, the three-choice scheme stabilizes at a slightly higher average gap (around 5) compared to the two-choice plot (around 4). This inversion does not appear in the non-batched results and, therefore, seems attributable to the batching effect, which diminishes the benefit of sampling additional bins and the extra choice does not translate into improved balancing and may even amplify fluctuations, leading to the observed anomaly.

Standard deviation

Figure 6 shows the corresponding standard deviation of these configurations in b-batched mode.

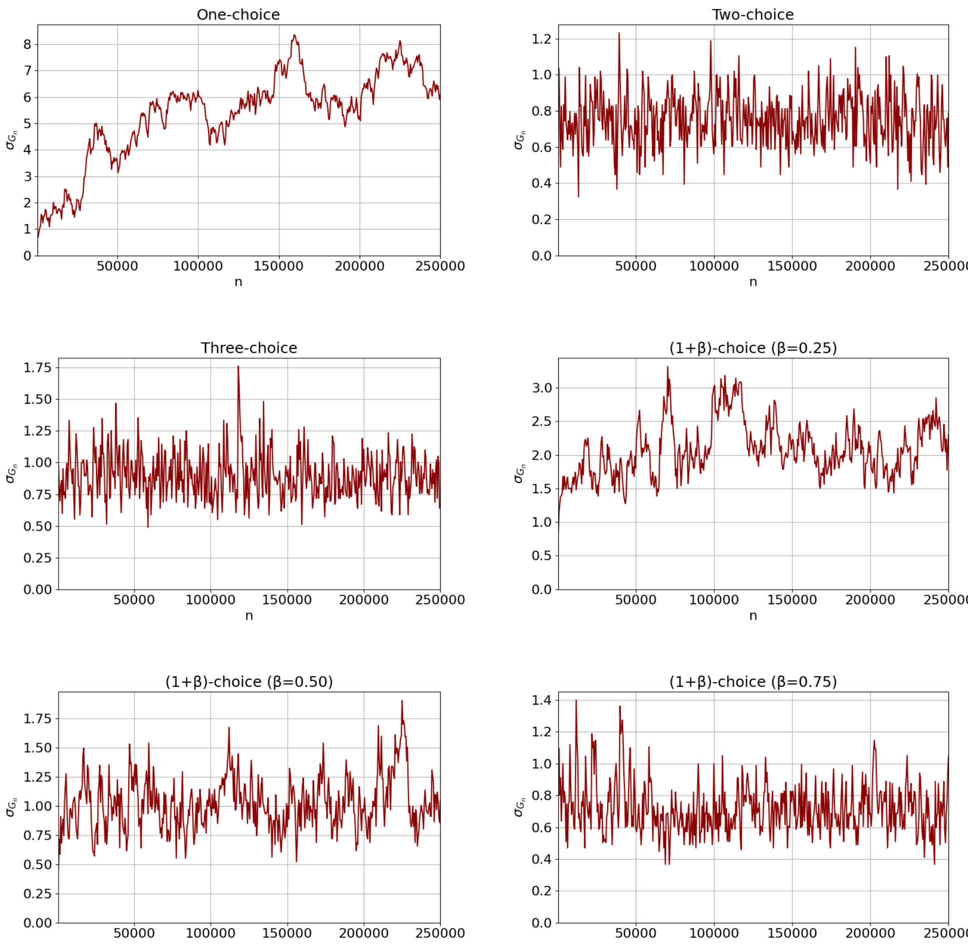

Figure 6: Evolution of the standard deviation of the gap for all previous configurations with b-batched setting.

As a general rule, the fluctuation is greater when the b-batched configuration is applied, and this is true in the cases where $d = 2$ or $d = 3$. Surprisingly, for the one-choice scheme, the standard deviation reaches slightly lower values than in its previous version, although it remains the worst.

The $(1 + \beta)$-choice curves show more uniform behavior because all three values of β converge to relatively low variability levels, in line with their non-batched values (between 1.5 and 3 for $\beta = 0.25$, around 1 for $\beta = 0.5$, and approximately 0.6–0.7 for $\beta = 0.75$). This indicates that, unlike the d-choice family, the $(1+\beta)$ scheme remains stable in batching and is less affected by the delayed feedback introduced by large batch sizes.

3.4 Partial information scheme

Figure 7 depicts the evolution of the average gap for the partial information scheme with the possibility of asking 1 or 2 questions, in both light-loaded and heavy-loaded scenarios.

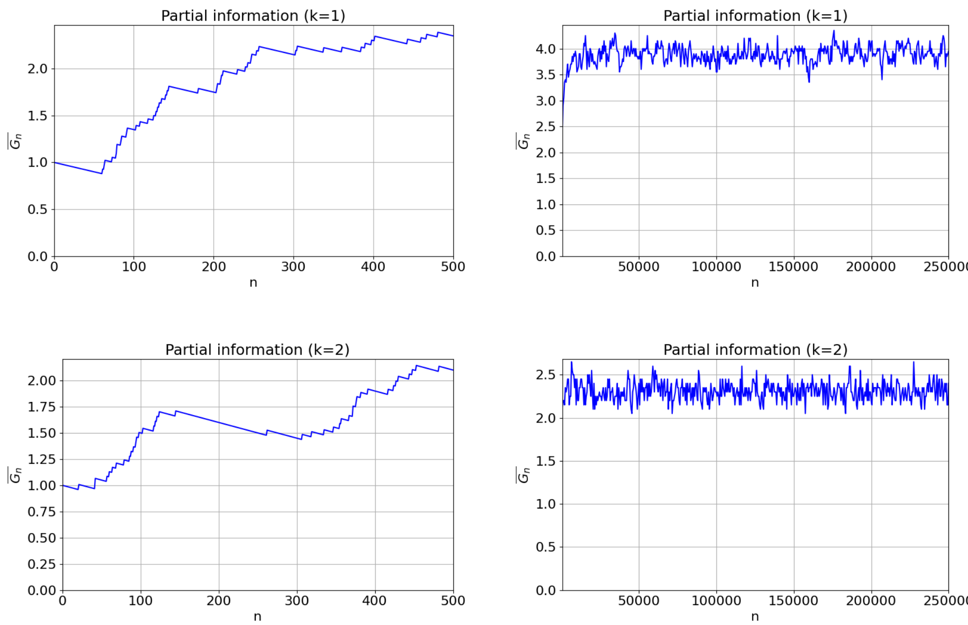

Figure 7: Evolution of the average gap for $k \in \{1, 2\}$ in light-loaded (left) and heavy-loaded (right) scenarios.

In the light-loaded case, the configuration with $k = 1$ shows a steady and relatively rapid increase in the gap, acting similarly to the single-choice scheme. When $k = 2$, the growth of the gap slows down considerably, even decreasing at some point as the number of balls increases. This demonstrates that even with limited information, increasing the number of quantile levels used in decision-making leads to a noticeably better distribution of load across bins.

In the heavy-loaded scenario, both configurations start from higher gap values due to the increased load per step, and the overall pattern is similar in both. The $k = 1$ version seems to have an average gap around 4, while $k = 2$ stabilizes at a lower level (around 2.3).

Compared to the $(1 + \beta)$ and d-choice schemes, the partial information curves exhibit very decent behavior, suggesting that this approach maintains a balance between performance and robustness under varying conditions.

Standard deviation

Figure 8 presents the corresponding standard deviation of the gap.

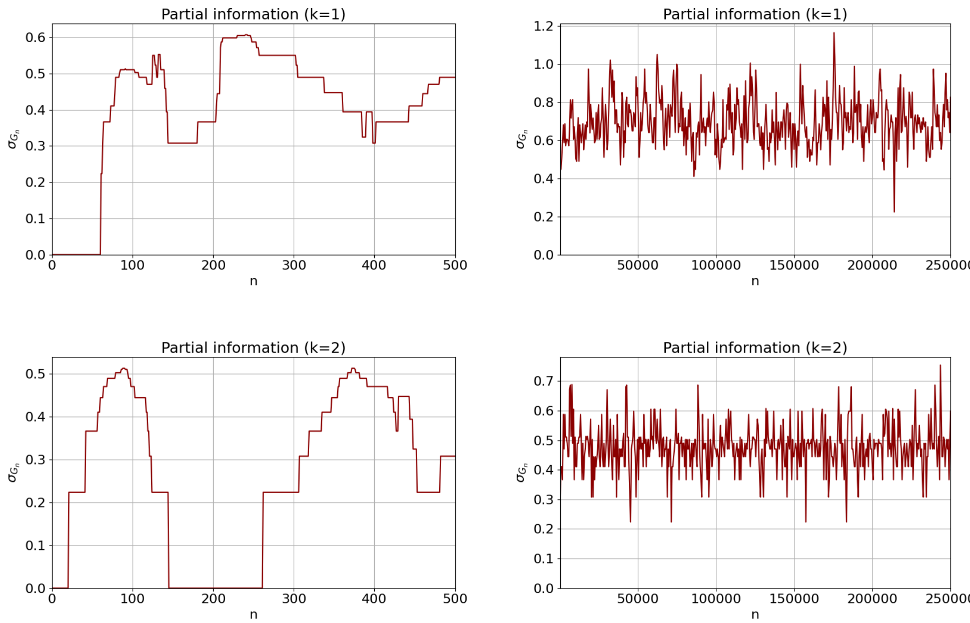

Figure 8: Evolution of the standard deviation of the gap for $k \in \{1, 2\}$ in light-loaded (left) and heavy-loaded (right) scenarios.

The variability follows the same qualitative evolution as the average gap, with $k = 1$ showing slightly larger fluctuations. The difference between both configurations is less dramatic than in the previous schemes, which indicates that, while additional information helps to improve load balance, the stochastic variability inherent to partial information still plays a relevant role.

4 Discussion

The results obtained across all experiments allow us to clearly identify differences in the performance of the allocation schemes and assess how the various parameters influence load balance. In general, simulations confirm that strategies that incorporate multiple choices or additional information about bin loads significantly reduce imbalance compared to the purely random one-choice scheme.

Among all schemes tested, the multi-choice variants consistently yield the best balance. The strategy of d-choice exhibits the strongest improvement as d increases: moving from $d = 1$ to $d = 2$ roughly halves the maximum observed gap and $d = 3$ further reduces it, though with diminishing returns. This suggests that the benefits of adding more choices saturate quickly, as each additional option provides a smaller marginal improvement once the system approaches equilibrium.

The $(1 + \beta)$-choice scheme achieves intermediate results between the one-choice and two-choice schemes. As β increases, the behavior smoothly transitions toward that of the two-choice process, with the gap decreasing correspondingly. This shows that the parameter β effectively controls the trade-off between simplicity and balance, allowing the scheme to adapt flexibly to the desired compromise between computational cost and performance.

The partial information scheme performs well relative to its simplicity. With $k = 1$, the scheme is not as good, but for $k = 2$, its performance approaches that of the two-choice algorithm. This shows that even limited access to load information can substantially improve balance, suggesting that partial visibility of the system state is sufficient to achieve near-optimal results under certain conditions.

Finally, the b-batched variant tends to worsen performance slightly compared to the non-batched cases. Batching introduces a delay between allocation decisions and their feedback, causing temporary imbalances and slower convergence toward equilibrium. Nevertheless, once a steady state is reached, the gap remains bounded and comparable to that of the corresponding non-batched configurations, indicating that the negative impact of batching is more pronounced during transient phases than in the final distribution.

In conclusion, the experiments demonstrate that allocating with even minimal additional information or choice can drastically improve load balance and that beyond a certain point, the marginal gain of adding more information diminishes. The most effective schemes are those that achieve this balance efficiently while maintaining low computational overhead.